

Robótica Móvil

Segundo cuatrimestre de 2025

Departamento de Computación - FCEyN - UBA

Introducción - clase 1

Conceptos preliminares - Introducción a Arduino

Organización de la Materia

Clases:

- ▶ Teóricas de 3 horas, miércoles de 10 a 13 hs en el aula 1102.
- ▶ Prácticas de 3 horas, viernes de 10 a 13 hs en el labo 1110.

Comunicación:

- ▶ La materia es **presencial**, todo lo que se diga/aclare en clase vale.
- ▶ Comunicación virtual a través del campus de la materia.

Evaluaciones y entregas:

- ▶ 13 Talleres (uno por cada clase práctica) que se realizan de manera individual o de a dos alumnos. Evaluación: Aprobado/Insuficiente.
- ▶ Si no llegan a aprobar un Taller en la clase correspondiente, pueden entregarlo en la siguiente (no más allá) o al final del cuatrimestre.
- ▶ Trabajo Práctico Final (código + informe) que se realiza de manera individual o de a dos alumnos. Evaluación: nota numérica (1 a 10).

Contenidos de la materia

En la materia vamos a ver:

- ▶ Cómo leer un sensor y controlar un motor con un microprocesador.
- ▶ Programación de microcontroladores (Arduino)
- ▶ Lectura de sensores y control de motores (CC).
- ▶ Comportamientos reactivos (por ejemplo, evasión de obstáculos)
- ▶ Cinemática de un robot móvil (diferencial, omnidireccional).
- ▶ Control de movimiento para el seguimiento de trayectorias.
- ▶ Localización probabilística (usando el Filtro Extendido de Kalman).
- ▶ Planificación de trayectorias (Algoritmos A* y RRT).
- ▶ Mapeo y localización simultánea (SLAM).
- ▶ Y más...

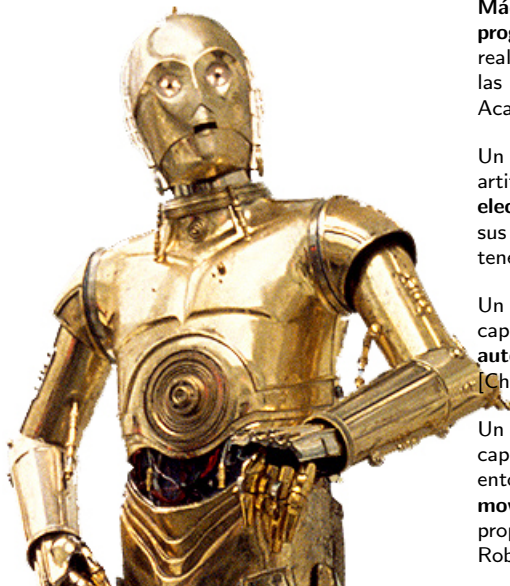
En la materia no vamos a ver:

- ▶ Diseño y construcción de robots (lo dejan para el *garage* de su casa).
- ▶ Aprendizaje automático (IA) aplicado a robots móviles.
- ▶ Robótica industrial (manipuladores).
- ▶ Visión por Computadora o Visión robótica.
- ▶ Robótica Colaborativa o Sistemas multi-robots.
- ▶ Interacción Persona-Máquina o Interacción Persona-Robot.
- ▶ Y más...(para la tesis?)

Bibliografía para el curso

- **Getting Started with Arduino**, Massimo Banzi, 3rd Edition, O'Reilly, 2014.
- **Introduction to autonomous mobile robots**, Siegwart, Roland, Illah Reza Nourbakhsh, Davide Scaramuzza. MIT press, 2011.
- **Probabilistic robotics**, Sebastian Thrun, Wolfram Burgard, Dieter Fox. MIT press, 2005.
- **Principles of robot motion: theory, algorithms, and implementation** Howie Choset, Kevin Lynch, Seth Hutchinson, George Kantor, Wolfram Burgard, Lydia Kavraki, Sebastian Thrun, MIT press, 2005.
- **A Concise Introduction to Robot Programming with ROS 2**, Francisco Martín Rico, Chapman & Hall, CRC Press, 2nd edition, 2024

¿Yo robot?



Máquina o ingenio electrónico **programable**, capaz de manipular objetos y realizar operaciones antes reservadas solo a las personas. [Diccionario de la Real Academia Española]

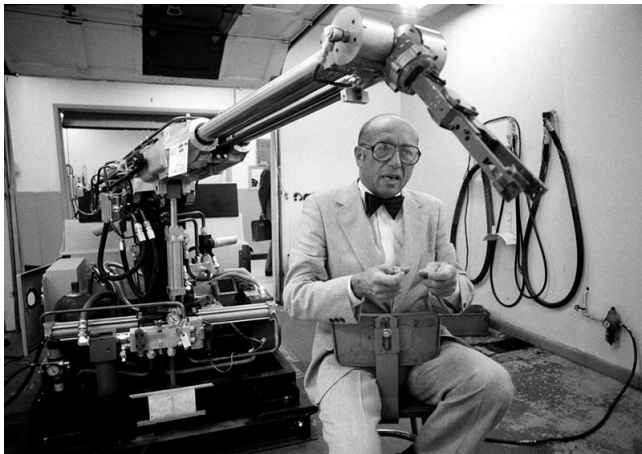
Un robot es una entidad virtual o mecánica artificial, por lo general un **sistema electromecánico** que, por su apariencia o sus movimientos, ofrece la sensación de tener un propósito propio. [Wikipedia]

Un robot es una máquina programable capaz de realizar tareas de **forma autónoma** o controlada por humanos. [ChatGPT]

Un robot "inteligente" es una máquina capaz de **extraer información** de su entorno y usar ese conocimiento para **moverse** en forma segura cumpliendo su propósito. [Ronald Arkin, "Behavior-Based Robotics" MIT Press, 1998]

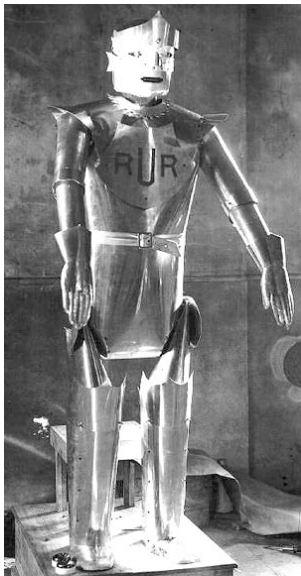
¿Yo robot?

I can't define a robot but I know one when I see one. [Joseph Engelberger]



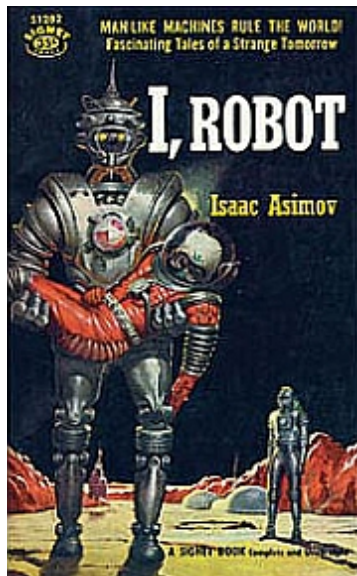
Joseph Frederick Engelberger fue físico e ingeniero, se lo considera el padre de la robótica industrial. Contruyó el primer robot industrial, el Unimate, en la década de 1950 y fundó la primera fábrica de robots: Unimation.

Mirando un poco hacia atrás



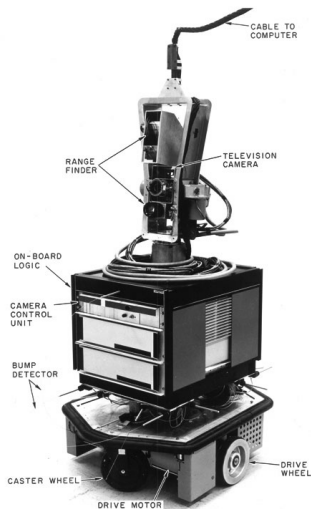
Representación de un robot en la obra de teatro R.U.R (Robots Universales Rossum) escrita por el checo Karel Čapek, que se estrenó en Teatro Nacional de Praga en 1921 y donde se acuñó el término.

De la ficción a la ciencia



Isaac Asimov propuso el término robótica (*robotics*) como palabra derivada de robot en su libro "Yo robot" en 1950.

El primer robot móvil autónomo



Robot Shakey, desarrollado por el Centro de Inteligencia Artificial del Instituto de Investigación de Stanford en 1966.

La Robótica Móvil

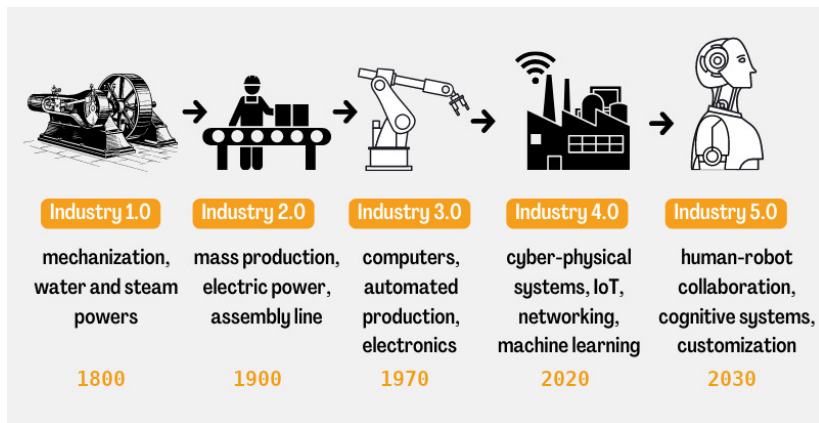
Un **robot móvil** tienen la capacidad de moverse en su entorno de forma segura, en contra posición a los **robots industriales** que se fijan a una ubicación de forma permanente.



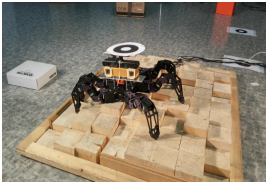
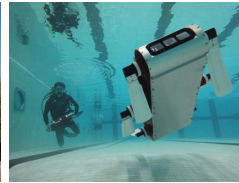
La Robótica Móvil hoy en día



El impacto de la Robótica Móvil



Algunos robots móviles y sus aplicaciones



Navegación autónoma

La navegación autónoma puede definirse a grandes rasgos como la capacidad de moverse de forma segura a lo largo de una trayectoria entre un punto de inicio y uno final. Para resolver el problema de la navegación autónoma, se tienen que contestar tres preguntas [1].

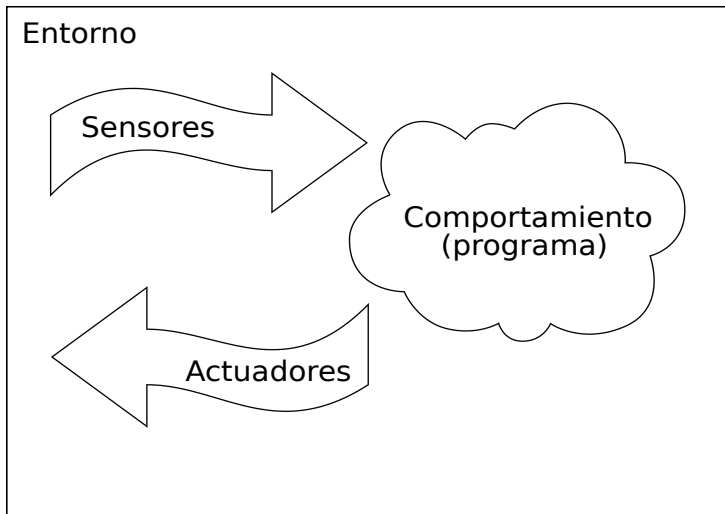
Pregunta:

- ¿Dónde estoy?
- ¿Por dónde estoy yendo?
- ¿Cómo llego hasta allí?

Respuesta:

- Cálculo de la posición (Localization)
- Representación del entorno (Mapping)
- Planeamiento de movimiento (Motion planning)

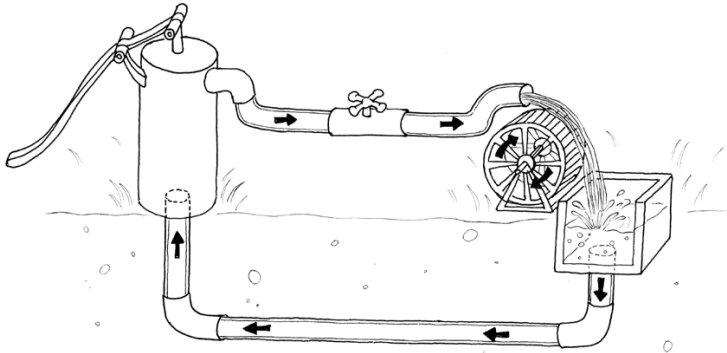
El ciclo de control: percibir, procesar, actuar!



Conceptos básicos de Electrónica - Introducción a Arduino



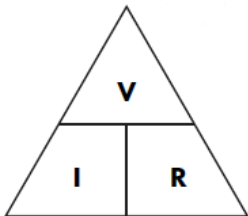
Conceptos básicos de electricidad



- ▶ La fuerza de la bomba determina la presión del agua. Es la tensión o diferencia de potencial del agua.
- ▶ La cantidad de agua que fluye por el caño es la cantidad de corriente o intensidad.
- ▶ La canilla y el ancho de los caños determina la resistencia al flujo de agua. La rueda también ejerce una resistencia al flujo de agua.

Ley de Ohm

Si mantenemos constante la presión del agua (tensión) y disminuimos el tamaño de los caños o cerramos la canilla (aumentamos la resistencia), disminuye el agua que fluye (corriente).



$$\mathbf{V = I \times R}$$

$$\mathbf{R = V/I}$$

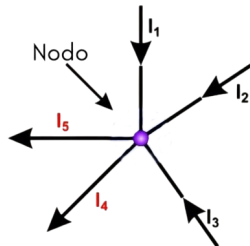
$$\mathbf{I = V/R}$$

- ▶ La unidad del tensión (mal dicho voltaje) es el volt (V).
- ▶ La de la corriente es el ampere (A).
- ▶ La de la resistencia es el ohm (Ω).

Leyes de Kirchhoff

Ley de Corrientes de Kirchhoff

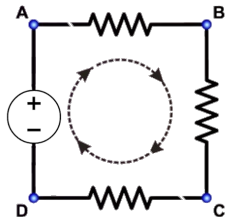
En cualquier nodo de un circuito, la suma algebraica de las corrientes es cero.



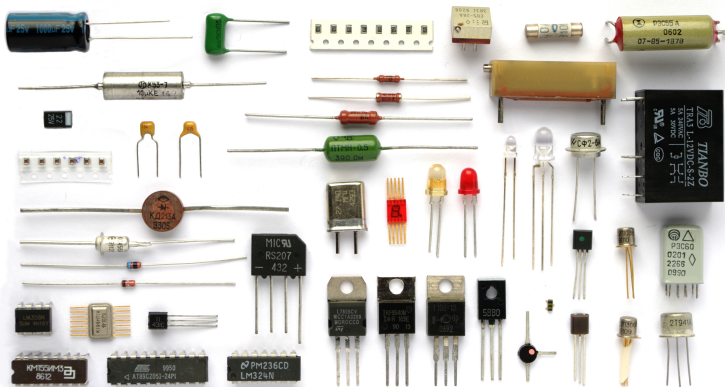
$$I_1 + I_2 + I_3 + (-I_4 - I_5) = 0$$

Ley de Voltajes de Kirchhoff

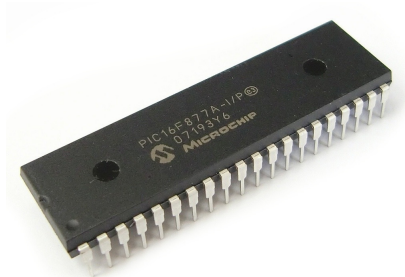
En cualquier lazo cerrado (malla) de un circuito, la suma algebraica de las diferencias de potencial (voltajes) es cero.



$$V_{AB} + V_{BC} + V_{CD} + V_{DA} = 0$$



Microcontroladores



- ▶ Circuito integrado programable.
- ▶ Arquitectura Hardvard (por lo general).
- ▶ Memoria de datos (SRAM) y memoria de programa (EEPROM, Flash).
- ▶ Pines de E/S .
- ▶ Módulos para control de periféricos y para comunicación.
- ▶ Varios modelos y marcas: PIC (Microchip), MSP430 (Texas Instruments), AVR (Atmel), Cortex-M (ARM).

Microcontrolador vs Microprocesador

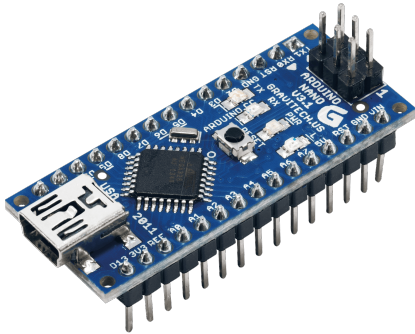
Microcontrolador

- Resuelve bien tareas específicas, e.g. lavarropas, microondas, robot, etc.
- Típicamente RISC de 8 o 16 bits con arquitectura Harvard.
- Los datos se almacenan en SRAM, el programa en EEPROM o Flash.
- Un programa que se repite indefinidamente.
- No tienen un Sistema Operativo (o a lo sumo tienen un RTOS)
- Memoria y E/S on board.
- Bajo costo, menor tamaño y poco consumo.

Microprocessor

- Concebidos para propósito general.
- Típicamente CISC de 32 o 64 bits con arquitectura Von Neuman.
- Los datos y los programas se almacenan en DRAM (DDR2-5).
- Muchos procesos corriendo en paralelo.
- Tienen Sistemas Operativos muy complejos.
- Memoria y E/S externos, por ejemplo placas de video, red, etc.
- Más caro, de mayor tamaño y consumo.

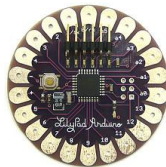
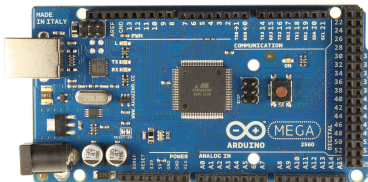
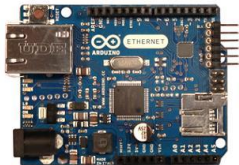
Arduino



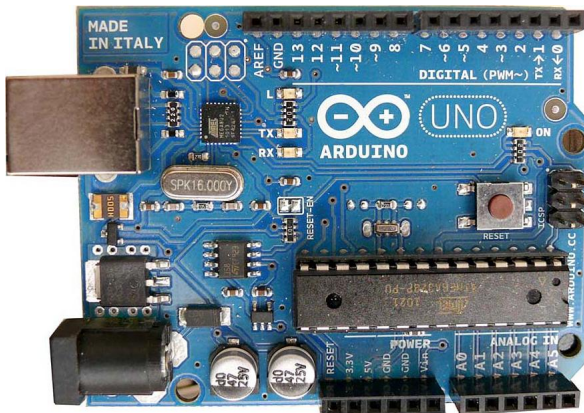
¿Qué es?

- Una empresa.
- Una placa (hardware).
- Una IDE (software).
- Open source.
- Programable por USB.
- Lenguaje de programación basado en Wiring/Processing
- Win-Mac-Linux.
- Barato.
- Muy popular.

Muchos modelos de Arduinos



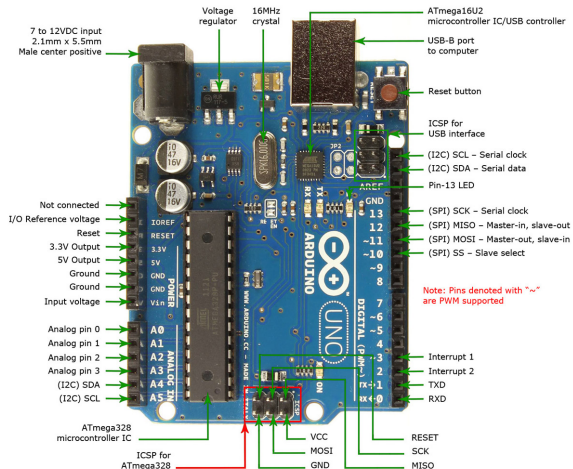
Arduino UNO



Especificaciones

- Basado en ATmega328.
- RISC, 8 bits, 1 MIPS.
- 32 KB Flash.
- 2 KB SRAM.
- 6 A/D 10 bits.
- 14 E/S digitales.

Esquema del Arduino UNO



¿Qué podemos hacer?

- E/S digital: controlar leds, switches, PWMs.
- E/S analógica: leer sensores.
- E/S serial, I2C, SPI: e.g. GPS.
- Con shields: muchas otras cosas más.

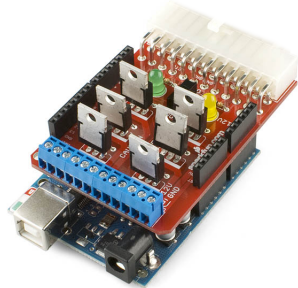
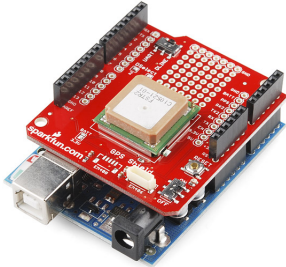
Terminología de Arduino

- ▶ Microcontrolador.
- ▶ Pines E/S.
- ▶ Shields
- ▶ Sensores y actuadores.
- ▶ Sketch
- ▶ Bibliotecas (mal dicho librerías) de funciones.

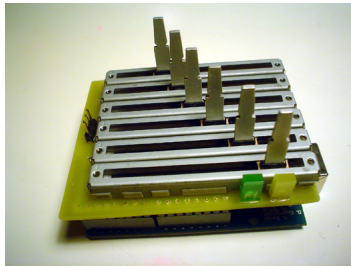
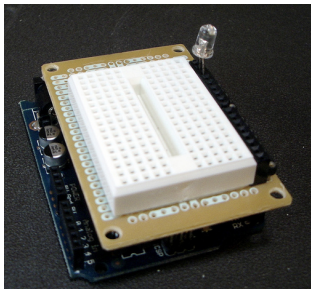
Pines (patitas) Entrada/Salida

- ▶ Salida digital: **escribe** un valor lógico (High o Low) en un pin.
- ▶ Entrada digital: **lee** un valor lógico (High o Low) de un pin.
- ▶ Entrada analógica: **lee** un valor analógico de un pin.
- ▶ Salida serial: **escribe** texto por consola.

Shields



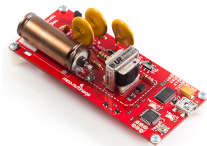
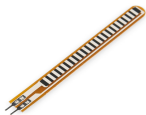
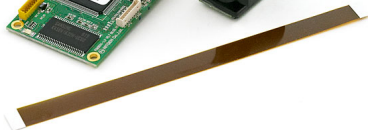
Shields



Shields



Sensores

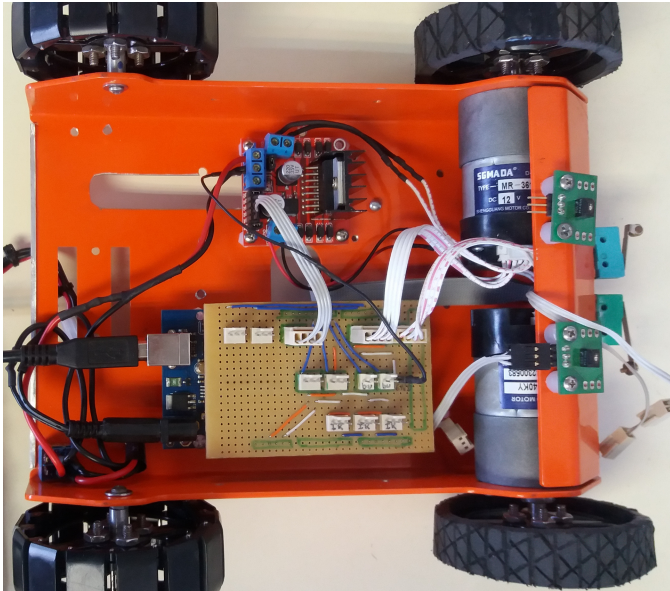


Kits Arduinos



A large collection of various electronic components is displayed on a white surface. The components include: several resistors of different values and colors; capacitors, including electrolytic and ceramic types; diodes, including LEDs and standard diodes; transistors; integrated circuits (ICs) of various packages; and other specialized components like a relay, a potentiometer, and a variable capacitor. The components are arranged in a somewhat organized manner, showing a wide variety of parts used in electronics.

Arduino en la vida real



¿Y el código dónde está?

- ▶ El código fuente (programa) listo para abrirse con el entorno de desarrollo integrado (IDE) de Arduino y ser cargado sobre nuestro dispositivo se llama `sketch`.
- ▶ Un sketch se compone de dos bloques que son requeridos para su compilación: `void setup()` y `void loop()`.
- ▶ La función `void setup()` se ejecuta una sola vez al comenzar el programa. Se usa para determinar si un determinado pin es de entrada o salida, establecer su valor inicial, inicializar el puerto serie, etc.
- ▶ La función `void loop()` se ejecuta de forma ininterrumpida, una y otra vez. Con este bucle logramos definir el comportamiento de nuestro Arduino, i.e, que nuestro programa responda ante los distintos eventos que se produzcan en nuestro entorno.
- ▶ **Ejemplo:** si el Arduino está programado para parpadear un LED, apenas lo enchufamos el programa empieza a correr y se ejecuta ese comportamiento hasta que lo apagamos.

Sketch

El siguiente sketch corresponde al comportamiento de parpadear un LED:

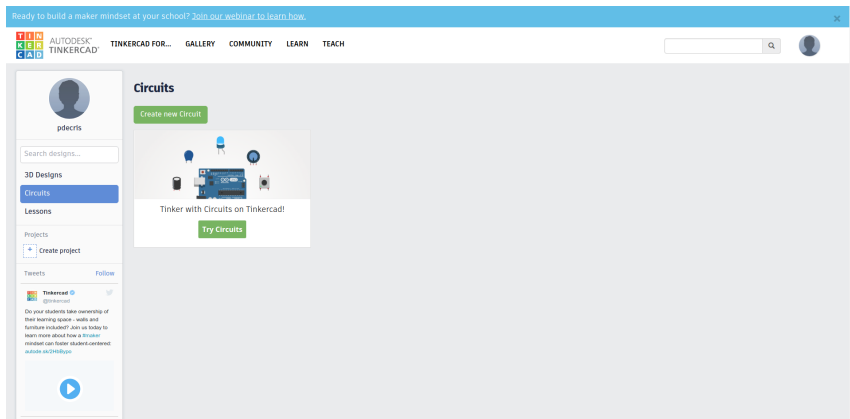
A screenshot of the Arduino IDE interface. The title bar reads "blinking_LED.ino | Arduino 1.6.10". The menu bar includes "File", "Edit", "Sketch", "Tools", and "Help". Below the menu bar is a toolbar with icons for opening, saving, and running. The main text area contains the following code:

```
/*  
 * Blink LED: turns on an LED for one second, then off for one second, repeatedly.  
 */  
  
// pin 13 has an LED connected on most Arduino boards. Give it a name.  
const int LED = 13;  
  
void setup() {  
  // put your setup code here, to run once:  
  pinMode(LED, OUTPUT); // initialize the digital pin as an output.  
}  
  
void loop() {  
  // put your main code here, to run repeatedly:  
  digitalWrite(LED, HIGH); // turn the LED on (HIGH is the voltage level)  
  delay(1000); // wait for a second;  
  digitalWrite(LED, LOW); //turn the LED off making the voltage LOW.  
  delay(1000); // wait for a second.  
}
```

The status bar at the bottom indicates "Arduino/Genuino Uno on /dev/ttyACM0".

Nota: por convención, todo Arduino tiene un LED asociado al pin 13.

Simulador Tinkercad



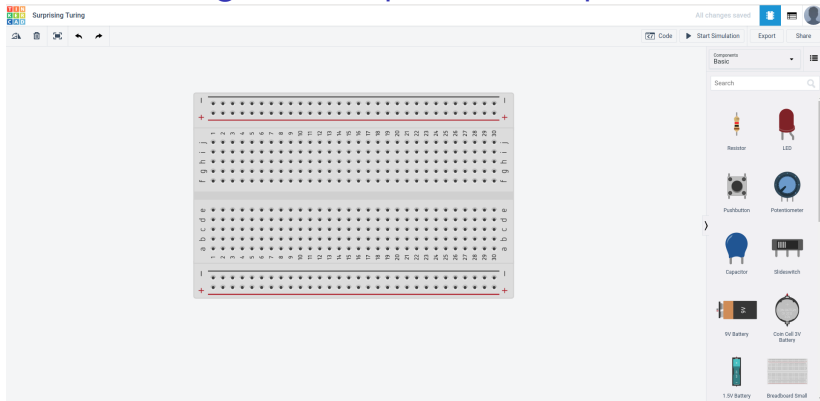
<https://www.tinkercad.com>

Simulador Tinkercad

- ▶ Simulación de circuitos electrónicos simples.
- ▶ Diseño 3D de piezas.
- ▶ Arduino (distintos modelos):
 - ▶ Programación! =D
 - ▶ Mucho más lento D=
- ▶ Comunidad: se pueden ver proyectos contriбуídos por otros

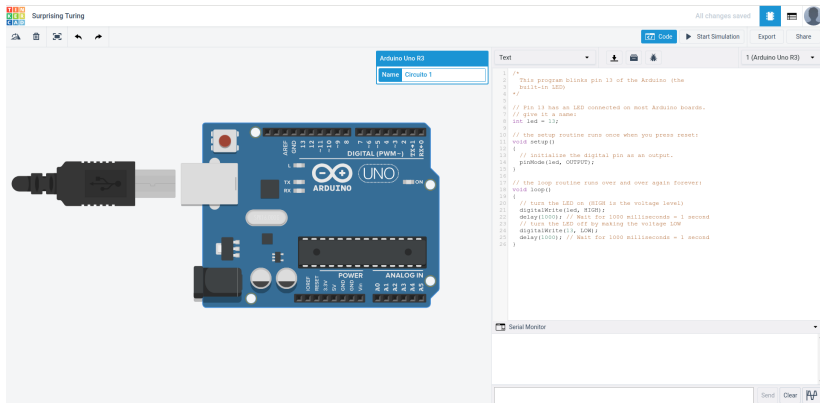
Importante: Vamos a usarlo en la materia (saquen una cuenta)

La simulación: algunos componentes, el protoboard



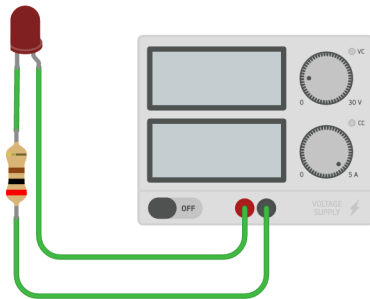
- ▶ Facilita las conexiones de componentes, sin soldar o hacer una placa.
- ▶ Tiene filas y columnas internamente interconectadas.
- ▶ 2 pares de líneas de alimentación independientes (positivo y tierra).
- ▶ 2 grupos de columnas de propósito general.

La simulación: Arduino, programación



(click para simulación)

Hola mundo! (version electrónica)

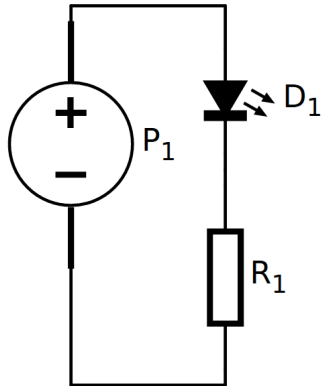


(click para simulación)

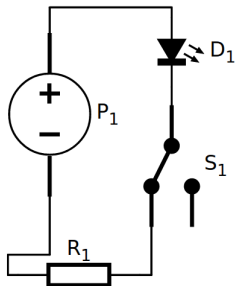
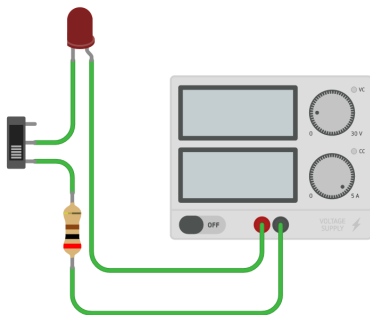
- ▶ Fuente: puedo elegir el voltaje que saca y limitar la corriente que ofrece
- ▶ LED: diodo emisor de luz. Podemos suponer que no ofrece resistencia (se “come” toda la corriente que puede).
- ▶ Resistencia: evita que el LED se queme (limita la corriente), lo elijo según el voltaje de la fuente (generalmente 5V)

Hola mundo! (esquemático)

- ▶ Diagrama formal para describir circuitos.
- ▶ Lectura directa y sencilla.
- ▶ Abstrae de ciertos detalles (ej: color del LED).



Tomemos el control (que el último apague la luz...)

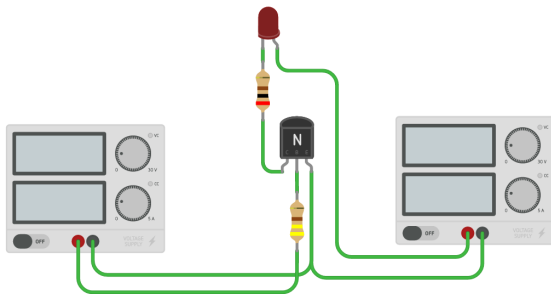


(click para simulación)

- ▶ Switch: según la posición de la llave, interconecta dos pines para cerrar el circuito.
- ▶ Control del LED: en una posición el circuito está abierto (no circula corriente), en la otra sí.

Un interruptor con lógica digital

Quiero controlar el LED con una señal *lógica* (un voltaje que sea 5V o 0V), no un interruptor mecánico (somos muy perezosos)



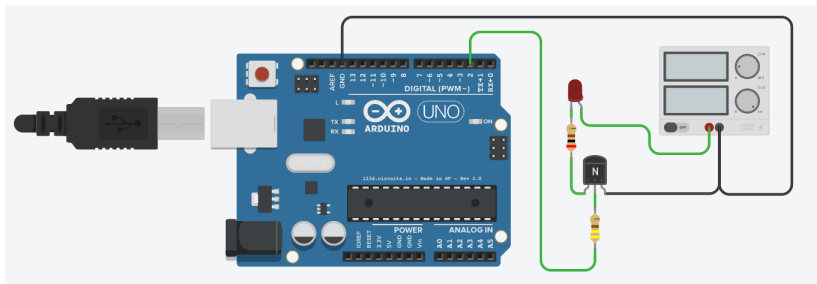
(click para simulación)

- ▶ Transistor (BJT): tiene tres pines, según la corriente que vea en el pin del medio, deja pasar la corriente por los otros dos pines.
- ▶ Segunda fuente: la usamos para simular la señal digital, la corriente que ofrece es mucho menor en comparación con la corriente que circula por el LED.

Un interruptor todavía más complicado

Cómo prender una lamparita, según un programador...

Usamos una señal de un microcontrolador para controlar el LED (o cualquier otra cosa!). Ahora el LED se prende solo!



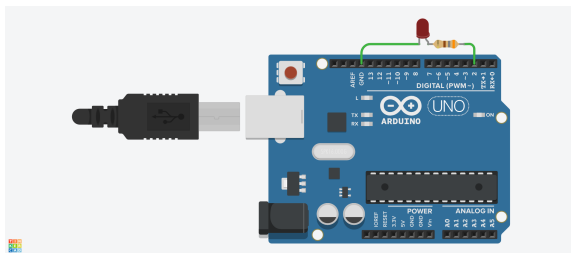
(click para simulación)

- Con Arduino podemos controlar el valor lógico del pin, que a su vez controla el circuito del LED + fuente.

Importante: notar que las masas (0V o *ground*) siempre van juntas. Los voltajes son relativos a una referencia que debe ser común!

Simplifiquemos un poco

El pin 3 del Arduino está controlado por un transistor, capaz de otorgar una corriente pequeña (10-40mA) que para el LED alcanza. Nos evitamos la fuente y el transistor.



(click para simulación)

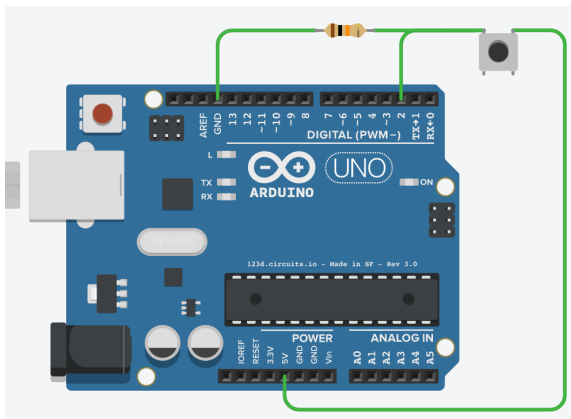
¿Y el código?

El Arduino está programado para parpadear el LED apenas lo enchufamos y el programa empieza a correr (sketch)

```
1 // el numero de PIN al que conectamos el LED
2 int led = 2;
3
4 // esta función corre una vez al inicio
5 void setup() {
6   pinMode(led, OUTPUT); /* configurar al pin como salida (vamos a sacar una señal por acá) */
7 }
8
9 // esta función corre como un ciclo infinito
10 void loop() {
11   digitalWrite(led, HIGH); // escribo una señal "HIGH" (5 volts)
12   delay(1000);             // espero un segundo
13   digitalWrite(led, LOW);  // escribo una señal "LOW" (0 volts)
14   delay(1000);             // espero un segundo
15 }
```

¿Cómo hago para leer una señal?

Un pin puede configurarse como “input”, para leer si hay 0 o 5 volts.

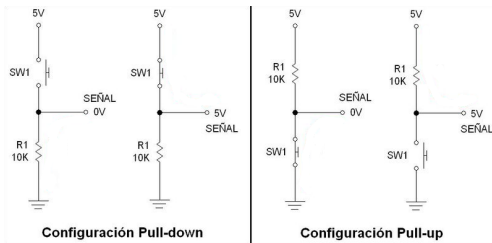


(click para simulación, ver código)

Pregunta: ¿por qué hay una resistencia?

Cómo evitar que el voltaje “flote”

- ▶ Cuando una entrada no está conectada a nada (boton sin apretar y sin cable + resistencia a GND) estaría leyendo “ruido”.
- ▶ Quiero que las entradas tengan valores siempre definidos: cableo la señal a tierra/positivo.



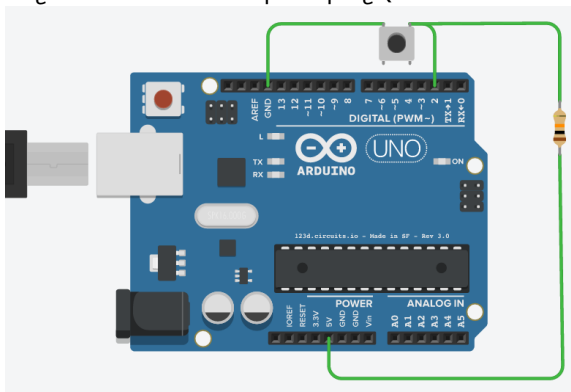
Y, entonces, ¿por qué la resistencia?

Respuesta corta: estaría poniendo en “corto” el circuito (evito pedir corriente excesiva a la fuente y “quemarla”)

Respuesta larga: quiero predeterminedar la señal en un valor conocido, pero que otra señal pueda tomar precedencia (lo que quiero leer)

Lectura digital, ¿y con un pull-up?

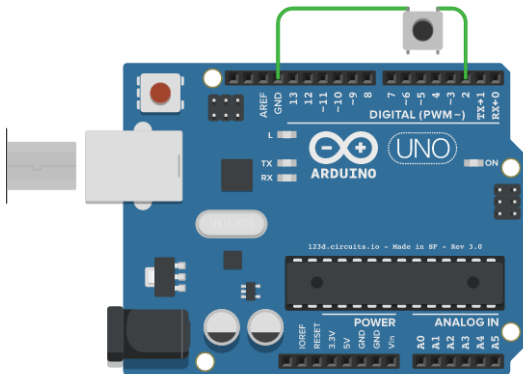
¿Cómo sería con un pull-up? ¿Qué cambiaría?



(click para simulación, ver código)

Lectura digital, ¿y más simple todavía?

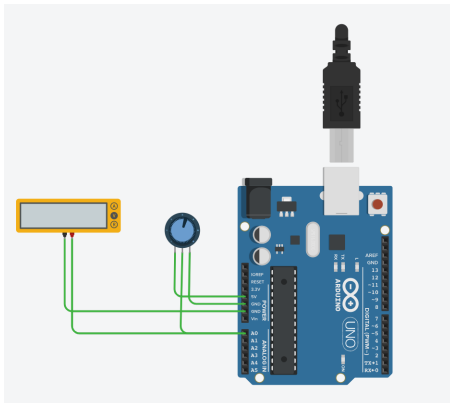
El Arduino en general tiene *pull-ups* internos.



(click para simulación, ver código)

No todo es digital: conversión A/D

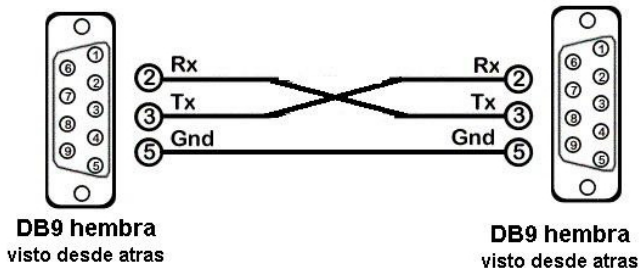
Existen pines especiales (A0-A5) que permiten leer voltajes con valores continuos (analógicos, no digitales) entre 0v y 5V. El valor en volts se convierte a un número binario de 10 bits (rango de 0 a 1023).



(click para simulación, ver código)

- Potenciómetro: resistencia *variable*, permite obtener un voltaje variable (no nos preocupemos por ahora cómo)

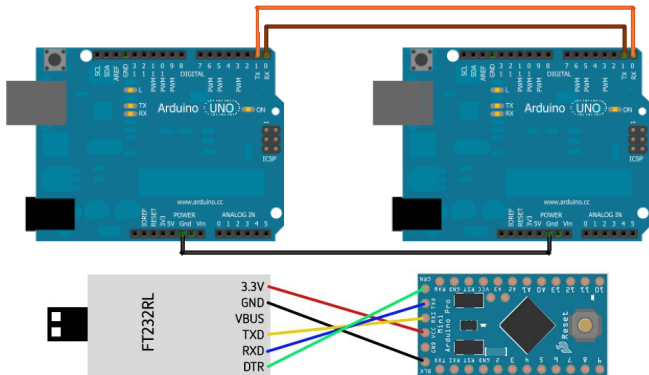
Comunicación serial: el afamado RS232



- ▶ RS232: Protocolo de comunicación serial (es decir, mando los bits de a uno por un cable), relativamente simple.
- ▶ Hardware: como mínimo RX, TX y GND (luego hay otras señales, pero no son estrictamente necesarias)
- ▶ Podemos usarlo para imprimir texto por consola en la PC (IDE de Arduino, Tinkercad), muy útil para depuración (*debugging*).

Comunicación serial en Arduino

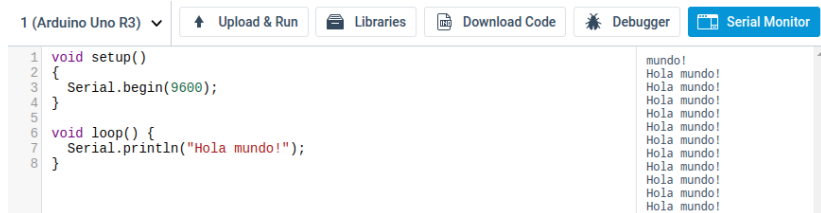
- ▶ Hardware serial: pines TX, RX que manejan las señales 0-5V
- ▶ Serial emulado: dispositivo USB que le hace creer a la PC que tiene un puerto serie (ejemplo: adaptador serial-usb)
- ▶ Conector USB Arduino (la PC ve al Arduino como un dispositivo serial)



Salida por terminal (consola) en Arduino

¿Cómo hacemos para imprimir desde el código (sketch) en Arduino?

- 1) Inicializar el puerto serie sobre USB en la función `void setup()`, indicando la velocidad de comunicación (baudios/bits por segundo)
- 2) Imprimir usando la función `Serial.print/println` (también puedo leer)



The screenshot shows the Arduino IDE interface. At the top, there is a toolbar with buttons for 'Upload & Run', 'Libraries', 'Download Code', 'Debugger', and 'Serial Monitor'. Below the toolbar, the sketch editor displays the following code:

```
1 void setup()  
2 {  
3   Serial.begin(9600);  
4 }  
5  
6 void loop() {  
7   Serial.println("Hola mundo!");  
8 }
```

To the right of the sketch editor, the Serial Monitor window is open, displaying the output of the code: "Hola mundo!". The output is repeated multiple times, indicating that the code is running in a loop.

Arduino: muy breve resumen de funciones

`analogRead(pin)` devuelve un numero entero de 10 bits (0-1023),
proporcional al voltaje del pin **analógico**

nota: $An \rightarrow \text{pin} = n$

`pinMode(pin, modo)` selecciona un pin **digital** como INPUT,
INPUT_PULLUP o OUTPUT

`digitalWrite(pin, valor)` escribe un valor HIGH o LOW en el pin
digital (poner como OUTPUT!)

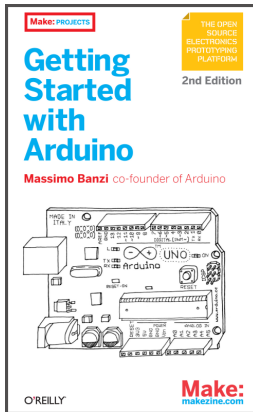
`digitalRead(pin)` devuelve un valor HIGH o LOW segun lo que se sense
en pin **digital** (poner como INPUT!)

`delay(ms)` pausa la ejecución durante el tiempo indicado en
milisegundos

`Serial.begin(velocidad)` configura el puerto serial (usar 9600 en el
simulador). Hacer en el setup!

`Serial.print` / `Serial.println` imprime (sin dejar y dejando un
salto de línea) un número, string, etc.

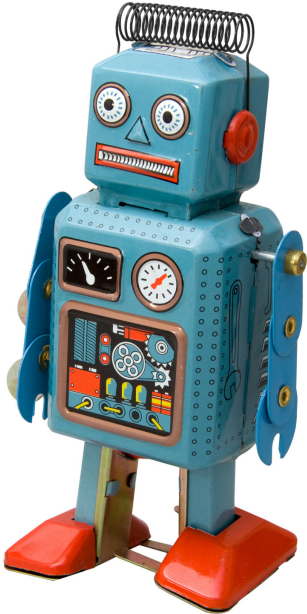
Más sobre Arduino



“Getting Started with Arduino”, Massimo Banzi, O'Reilly, Segunda Edición, 2014.

Página oficial del proyecto Arduino:
<https://www.arduino.cc/>

Para la próxima clase... basta de tanta zaraza



- ▶ Sacar un usuario en Tinkercad: <https://www.tinkercad.com/>
- ▶ Repasar los conceptos y circuitos de esta clase.
- ▶ Bajar la consigna del Taller 1 desde el campus.
- ▶ Traer auriculares y/o parlantitos (haremos ruido!).